Path-Coloring Algorithms for Plane Graphs

Aven Bross
Department of Computer Science
University of Alaska
Fairbanks, AK 99775-6670
dabross@alaska.edu

Glenn G. Chappell
Department of Computer Science
University of Alaska
Fairbanks, AK 99775-6670
chappellg@member.ams.org

Chris Hartman
Department of Computer Science
University of Alaska
Fairbanks, AK 99775-6670
cmhartman@alaska.edu

October 1, 2026

2010 Mathematics Subject Classification. Primary 05C38; Secondary 05C10, 05C15.
Key words and phrases. Path coloring, list coloring, algorithm.

Abstract

A path coloring of a graph G is a vertex coloring of G such that each color class induces a disjoint union of paths. We present two efficient, linear-time algorithms to construct a path coloring of a plane graph.

The first algorithm, based on a proof of Poh, is given a plane graph, and produces a path coloring of the given graph using three colors.

The second algorithm, based on similar proofs by Hartman and Škrekovski, performs a list-coloring generalization of the above. The algorithm is given a plane graph and an assignment of lists of three colors to each vertex, and it it produces a path coloring of the given graph in which each vertex receives a color from its list.

Implementations are available for all algorithms that are described.

1 Introduction

All graphs will be finite and simple. See West [12] for graph theoretic terms.

A graph is *planar* if it may be drawn in the plane without edge crossings, and such a drawing is called an *embedding*. A *plane graph* is a planar graph with a fixed embedding. Subgraphs of a plane graph will themselves be plane graphs with the inherited embedding.

A *path coloring* is a (vertex) coloring where each color class induces a disjoint union of paths. A graph G is *path k-colorable* if G admits a path k-coloring (a path coloring using at most k colors). A *list assignment* L of a graph G maps each $v \in V(G)$ to a finite set of colors $L(v)$. An *L-list-coloring* is a coloring c such that $c(v) \in L(v)$ for each $v \in V(G)$. A graph is *path k-choosable* if it admits a path L-list-coloring for every list assignment L with at least k colors in each list, i.e. $|L(v)| \geq k$ for each $v \in V(G)$.

Broere & Mynhardt conjectured [2, Conj. 16] that every planar graph is path 3-colorable. This was proven independently by Poh [9, Thm. 2] and Goddard [6, Thm. 1].

Theorem 1.1 (Poh 1990, Goddard 1991)**.** *If G is a planar graph, then G is path 3-colorable.* □

It is easily shown that the "3" in Theorem 1.1 is best possible. In particular, Chartrand & Kronk [5, Section 3] gave an example of a planar graph whose vertex set cannot be partitioned into two subsets, each inducing a forest.

Hartman [7, Thm. 4.1] proved a list-coloring generalization of Theorem 1.1 (see also Chappell & Hartman [4, Thm. 2.1]). Essentially the same coloring technique was used by Škrekovski [11, Thm. 2.2b] to prove a result slightly weaker than Theorem 1.2.

Theorem 1.2 (Hartman 1997)**.** *If G is a planar graph, then G is path 3-choosable.* □

We discuss two path coloring algorithms based on proofs of the above theorems.

In Section 2 we outline our graph representations and the basis for our computations of time complexity.

Section 3 covers an algorithm based on Poh's proof of Theorem 1.1. The algorithm is given a plane graph; it produces a path coloring of the given graph using three colors.

Section 4 covers an algorithm based on Hartman's proof of Theorem 1.2, along with the proof of Škrekovski mentioned above. The algorithm is given a plane graph and an assignment of a list of three colors to each vertex; it produces a path coloring of the given graph in which each vertex receives a color from its list.

Section 5 provides performance results for our implementations of each algorithm [3]. The section also discusses how each algorithm may be modified to benefit from parallelism.

2 Graph Representations and Time Complexity

A *face* of a plane graph is a subgraph that, as drawn in the embedding, forms the border of a region of the plane that contains no point of the embedding. A plane graph is *triangulated* if each face is a 3-cycle, and *weakly triangulated* if it is connected and each interior (bounded) face is a 3-cycle.

A *rotation scheme* for a graph is a circular ordering of the incident edges around each vertex. An embedding induces a rotation scheme by the clockwise (or counter-clockwise) order in which edges appear around each vertex. The induced rotation scheme describes all of the combinatorial properties of an embedding. For example, the faces of the embedding correspond to the closed walks induced by the rotation scheme: for $u \in V(G)$, $uv \in E(G)$

the induced closed walk follows uv from u to v, then follows from v the edge subsequent to uv in the rotation scheme around v, and so on, until reaching uv again from u.

The input to each algorithm we'll discuss will be a weakly triangulated plane graph G with $n = |V(G)|$ vertices and $m = |E(G)|$ edges. The input size will be n, but it is equivalent to take m to be the input size: if $n \geq 3$, then $m \leq 3n - 6$ by Euler's formula. The output of each algorithm will be a path coloring of G.

A connected planar graph may be embedded (by finding a rotation scheme induced by an embedding) and triangulated (by adding edges across faces that are not 3-cycles) in $\mathcal{O}(n)$ time, see e.g. Hopcroft and Tarjan [8] or Boyer and Myrvold [1]. Therefore, since the removal of any edges that were added to produce a triangulation will not invalidate the output path coloring, we may consider each algorithm to take an arbitrary planar graph with n vertices as its input, with no change to the time complexity.

We will represent the input plane graph G via *adjacency lists*: a list, for each vertex v, of the neighbors of v. Each list will be ordered according to the counter-clockwise rotation scheme induced by G's embedding. A vertex will be represented by an integer $0 \ldots n-1$. In algorithms, $N(v)$ will refer to v's adjacency list.

Let $v \in G$ and $u \in N(v)$. Define $\text{next}_v(u) \in N(v)$ to be the neighbor counter-clockwise from u around v. In algorithms, ${}^v u$ will be a reference to u's entry in the adjacency list $N(v)$. In this way, ${}^v u$ represents both u and the relative position of the edge uv around v in the embedding. Define $\text{next}({}^v u) = {}^v(\text{next}_v(u))$ to be the next entry in the list $N(v)$.

The $\mathcal{O}(n)$ time complexity of the algorithm in Section 4 depends on our ability to compute a *back reference* $\text{back}({}^v u) = {}^u v$ in $\mathcal{O}(1)$ time. An *augmented adjacency list* stores a pair $(u, {}^u v)$ for each neighbor u of v. Given an adjacency list representation, an augmented representation may be built in $\mathcal{O}(m)$ time, see Algorithm 2.1.

Algorithm 2.1: *Augment*(N)

Input. An adjacency list representation N of a graph G.

Output. An augmented adjacency list representation N^* of G with the same rotation scheme as N.

Step 1. For each $v \in G$, construct an augmented adjacency list $N^*(v)$ with the same ordering as $N(v)$, leaving the ${}^u v$ portion of each entry $(u, {}^u v)$ in $N^*(v)$ uninitialized.

Step 2. For each vertex v, construct a list $\text{Wrk}(v)$ of vertex-reference pairs with capacity $\deg(v)$. For each v from 0 to $n-1$, iterate through $N^*(v)$. For each neighbor u in $N^*(v)$, append the pair $(v, {}^v u)$ to $\text{Wrk}(u)$, where ${}^v u$ is a reference to u's entry $N^*(v)$.

Step 3. For each v from $n-1$ to 0, iterate through $\text{Wrk}(v)$. Upon reaching a pair $(u, {}^u v)$ in $\text{Wrk}(v)$, the last element of $\text{Wrk}(u)$ will be $(v, {}^v u)$; see below. Lookup the entry ${}^v u$ in $N^*(v)$ and initialize the back reference ${}^u v$, and vice versa. Pop $(v, {}^v u)$ from $\text{Wrk}(u)$.

After completing Step 2 in Algorithm 2.1 each list $\text{Wrk}(v)$ will contain $(u, {}^u v)$ for each neighbor u of v, sorted in increasing order by u. Suppose that v is the current vertex at a given iteration of the loop in Step 3. For each edge $uw \in E(G)$ such that $u < w$ and $v < w$, prior iterations of the loop will have initialized the back references for $(w, {}^w u)$ in $N^*(u)$ and $(u, {}^u w)$ in $N^*(w)$, and removed $(w, {}^w u)$ from $\text{Wrk}(u)$. Therefore, for each $(u, {}^u v)$ in $\text{Wrk}(v)$, the list $\text{Wrk}(u)$ will only contain entries $(w, {}^w u)$ where $w \leq v$.

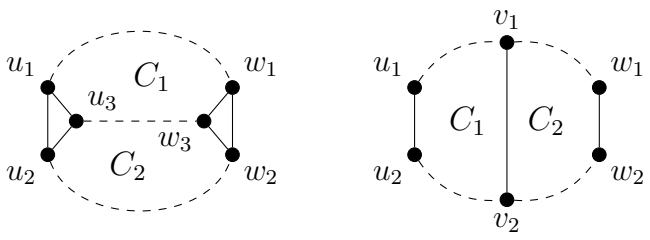


Figure 1: The proof of Lemma 3.1 when C is induced (left), or has a chord (right).

3 Path Coloring

In this section we describe a linear time algorithm to path 3-color a plane graph, based on Poh's constructive proof of Theorem 1.1.

Let C be a cycle in a connected plane graph G. If $u, v \in C$, then we define $C[u, v]$ to be the clockwise u, v-path along C; if $u = v$, then $C[u, u] = u$. Define $\overline{\text{int}}(C)$ to be the subgraph of G consisting of C and all vertices and edges interior to C; equivalently, $\overline{\text{int}}(C)$ is the maximal subgraph of G with outer face C. Define $\text{int}(C) = \overline{\text{int}}(C) - C$.

Let P be a path in a plane graph G. If $u, v \in P$, then we define $P[u, v]$ to be the u, v-path in P. We also define $\overline{\text{int}}(P) = P$ and $\text{int}(P)$ to be empty.

Lemma 3.1 (Poh 1990)**.** *Let G be a biconnected, weakly triangulated plane graph with outer cycle C. Let $c : V(C) \to S \subsetneq \{1, 2, 3\}$ be a 2-coloring of C such that each color class induces a nonempty path. There exists an extension of c to a path 3-coloring $c : V(G) \to \{1, 2, 3\}$ such that for each $v \in G - C$, if $vu \in E(G)$ with $u \in C$, then $c(v) \neq c(u)$.*

Proof. If $|V(G)| \leq 3$, then $G = C$. We proceed by induction on the order of G.

Suppose that $G \neq C$. Let P_1, P_2 be the two paths induced by the 2-coloring of the outer face C. We may write $P_1 = C[u_1, w_1]$ and $P_2 = C[w_2, u_2]$ for vertices $u_1, u_2, w_1, w_2 \in C$.

Suppose that C is an induced subgraph of G. Take $u_3, w_3 \in G - C$ such that the cycles u_3, u_2, u_1 and w_3, w_1, w_2 are faces of G; note that u_3 and w_3 are uniquely determined, but it may be that $u_3 = w_3$. Since C is induced, $G - C$ is connected. Let P_3 be an induced u_3, w_3-path in $G - C$. Color each vertex of P_3 with $d \in \{1, 2, 3\} - S$. Let $C_1 = P_1 \cup P_3 \cup u_3u_1 \cup w_3w_1$ and $C_2 = P_2 \cup P_3 \cup u_3u_2 \cup w_3w_2$, see Figure 1 (left). By the inductive hypothesis, the 2-coloring of C_1 extends to a path 3-coloring of $\overline{\text{int}}(C_1)$. Similarly, the coloring of C_2 extends to a path 3-coloring of $\overline{\text{int}}(C_2)$. Together these colorings form a 3-coloring of G. Since C, P_3, $\text{int}(C_1)$, and $\text{int}(C_2)$ partition $V(G)$, and no vertex shares a color with a neighbor in a different partition, this 3-coloring is a path coloring satisfying the lemma.

Suppose that C is not an induced subgraph. Then there exists an edge $v_1v_2 \in E(G) - E(C)$ with $v_1 \in P_1$ and $v_2 \in P_2$. Let $C_1 = C[v_2, v_1] \cup v_1v_2$ and $C_2 = C[v_1, v_2] \cup v_1v_2$, see Figure 1 (right). By the inductive hypothesis, c extends to path 3-colorings of $\overline{\text{int}}(C_1)$ and $\overline{\text{int}}(C_2)$. Together these colorings form a path 3-coloring of G satisfying the lemma. □

Let G be a triangulated plane graph. We may trivially path 2-color the outer triangle. By Lemma 3.1 this extends to a path 3-coloring of G. If G is an arbitrary planar graph,

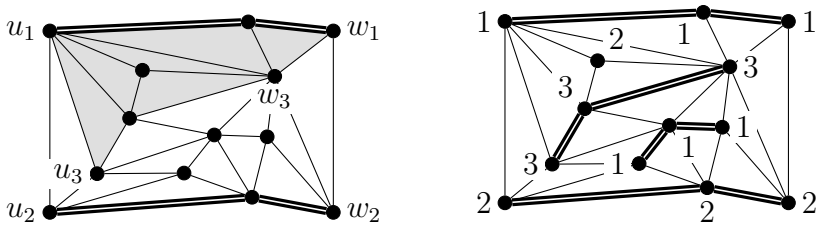


Figure 2: An example setup for Poh's algorithm highlighting the subgraph G_1 induced by $V(P_1) \cup N(P_1) - V(P_2)$ (left), and a path 3-coloring produced by Algorithm 3.2 (right).

we may compute an embedding and add edges to produce a triangulated plane graph G'. Theorem 1.1 follows immediately as any path coloring of G' is also a path coloring of G.

Let G, C, P_1, and P_2 be as in the proof of Lemma 3.1, and assume C is induced in G. For a subgraph H of G, let $N(H)$ be the set of vertices in $G - H$ that have a neighbor in H. Let G_1 be the subgraph of $\overline{\text{int}}(C)$ induced by $V(P_1) \cup N(P_1) - V(P_2)$, see Figure 2. Let C_1 be the outer face of G_1. Algorithm 3.2 implements Poh's algorithm in $\mathcal{O}(n)$ time by using $P_3 = C_1 - P_1 = C_1[w_3, u_3]$ as the u_3, w_3-path. Observe that P_3 is induced in G: G_1 is induced, so there is no chord of P_3 exterior to C_1, and each $v \in P_3$ has a neighbor in P_1, so there is no chord of P_3 interior to C_1. To compute P_3 in the algorithm: we'll mark the vertices in G_1, then walk counter-clockwise from u_3 to w_3 along the outer face C_1.

Define $p_1 : N(P_1) \to V(P_1)$ to be the map sending each $v \in N(P_1)$ to its neighbor in $P_1 = C[u_1, w_1]$ that is closest to w_1, i.e. $V(C[p_1(v), w_1]) \cap N(v) = \{p_1(v)\}$.

Algorithm 3.2: *Path3Color*$(u_1, {}^{u_1}u_2, \ell_1)$

Input. Let C be an induced cycle in a weakly triangulated plane graph G represented via adjacency lists. Let c be a length n array representing a 2-coloring of C such that each color class induces a nonempty path, respectively $P_1 = C[u_1, w_1]$ and $P_2 = C[w_2, u_2]$, where $u_1, u_2, w_1, w_2 \in C$. Assume that $c(v) = 0$ for each $v \in \text{int}(C)$. Let S be a length n array, and ℓ_1 an integer, such that for $v \in \text{int}(C)$, $S(v) = \ell_1$ if and only if $v \in N(P_1)$.

The concrete input will be G, c, and S, along with the tuple $(u_1, {}^{u_1}u_2, \ell_1)$.

Output. For each vertex $v \in \text{int}(C)$, a color in $\{1, 2, 3\}$ will be assigned to $c(v)$ such that c represents a path 3-coloring of $\overline{\text{int}}(C)$ extending the original 2-coloring of C. Moreover, if $v \in \text{int}(C)$ has a neighbor $u \in C$, then $c(v) \neq c(u)$.

Procedure. Let ${}^{u_1}u_3 := \text{next}({}^{u_1}u_2)$. It may be useful to refer to Figures 2 and 3.

Base case. If $c(u_3) \neq 0$, then $u_3 = w_2$ or $u_3 = w_1$. Thus C is a face and we are done.

Recursive case. Suppose that $c(u_3) = 0$, i.e. $u_3 \in \text{int}(C)$. Let $\ell_3 := \max_{v \in G} S(v) + 1$. Let $d \in \{1, 2, 3\} - \{c(u_1), c(u_2)\}$. We'll walk the induced u_3, w_3-path $P_3 = C_1 - P_1$, where C_1 is the outer face of the subgraph G_1 of $\overline{\text{int}}(C)$ induced by $V(P_1) \cup N(P_1) - V(P_2)$.

Step 1. Let $a := u_1$ and $z := u_3$; here z stores the current vertex of P_3, and a stores the vertex clockwise from z around C_1. Let $t :=$ NULL and ${}^t x :=$ NULL; here t and ${}^t x$ store the last edge tx visited where $t \in P_3$ and $x \in P_2$ (the first such edge will be u_3u_2).

Loop 2. We'll walk counter-clockwise along C_1, coloring one vertex z per iteration, and halt after $z = w_3$. Color $c(z) \leftarrow d$. Find ${}^z a$ in $N(z)$ and let ${}^z v := \text{next}({}^z a)$. Let

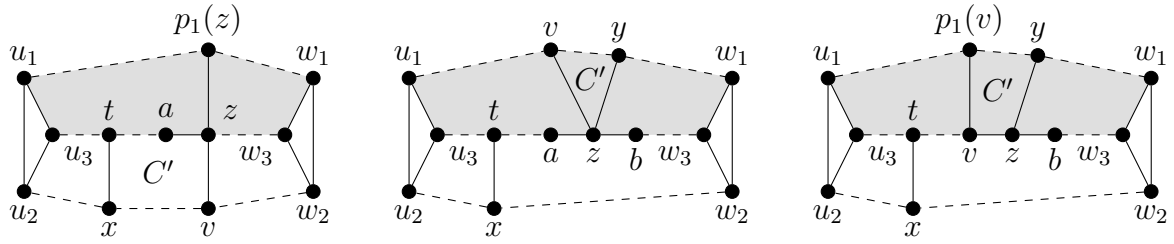


Figure 3: A sketch of Case 2.1.2 (left), Case 2.2.2 (middle), and Case 2.2.3 (right) in Algorithm 3.2. In each sketch, the highlighted region represents G_1.

$b :=$ NULL; here b will store the vertex in P_3 counter-clockwise from z around C_1, if such a vertex exists. Let ${}^zy :=$ NULL; here ${}^zy$ will store the last edge zy visited where $y \in P_1$.

Loop 2.1. We'll first consider the neighbors of z from $\text{next}_z(a)$ to $p_1(z)$ counter-clockwise. Each iteration will handle one neighbor ${}^zv$. We'll make a recursive call on each induced cycle C' in G that satisfies $V(C') \subseteq V(P_2) \cup V(P_3)$ and $C' \cap P_3[z, w_3] = z$.

Case 2.1.1. Suppose that $c(v) = 0$, i.e. $v \in \text{int}(C)$. If $S(v) = \ell_1$ and $b =$ NULL, then $v \in P_3$ and we set $b \leftarrow v$. Set $S(v) \leftarrow \ell_3$ and ${}^zv \leftarrow \text{next}({}^zv)$. Repeat Loop 2.1.

Case 2.1.2. Suppose $c(v) = c(u_2)$, i.e. $v \in P_2$. If $t \neq$ NULL, call *Path3Color*$(t, {}^tx, \ell_3)$ to color $\overline{\text{int}}(C')$, where $C' = P_3[t, z] \cup C[v, x] \cup tx \cup zv$; refer to Figure 3 (left). Set $t \leftarrow z$, ${}^tx \leftarrow {}^zv$, and ${}^zv \leftarrow \text{next}({}^zv)$. Repeat Loop 2.1.

Case 2.1.3. Suppose that $c(v) = c(u_1)$, i.e. $v = p_1(z) \in P_1$. Break Loop 2.1. If $v = a$, skip to Step 2.3. Otherwise, set ${}^zy \leftarrow {}^zv$, ${}^zv \leftarrow \text{next}({}^zv)$, and proceed to Loop 2.2.

Loop 2.2. We'll now consider the neighbors of z from $\text{next}_z(p_1(z))$ to a counter-clockwise. Each iteration will handle one neighbor ${}^zv$. We'll make a recursive call on each induced cycle C' that satisfies $V(C') \subseteq V(P_1) \cup V(P_3)$ and $C' \cap P_3[z, w_3] = z$.

Case 2.2.1. Suppose that $c(v) = 0$, i.e. $v \in \text{int}(C_1)$. Assign $S(v) \leftarrow \ell_3$ and ${}^zv \leftarrow \text{next}({}^zv)$. Repeat Loop 2.2.

Case 2.2.2. Suppose that $c(v) \neq 0$ and $v \neq a$, i.e. $v \in P_1$. Call *Path3Color*$(z, {}^zy, \ell_3)$ to color $\overline{\text{int}}(C')$, where $C' = C[v, y] \cup z \cup zy \cup zv$; see Figure 3 (middle). Set ${}^zy \leftarrow {}^zv$ and ${}^zv \leftarrow \text{next}({}^zv)$, then repeat Loop 2.2.

Case 2.2.3. Suppose $v = a$. Call *Path3Color*$(z, {}^zy, \ell_3)$ to color $\overline{\text{int}}(C')$, where $C' = C[p_1(v), y] \cup P_3[v, z] \cup vp_1(v) \cup zy$; see Figure 3 (right). Break Loop 2.2 and go to Step 2.3.

Step 2.3. If $b \neq$ NULL, then assign $a \leftarrow z$, $z \leftarrow b$, and repeat Loop 2. Otherwise, $z = w_3$ and c is a path 3-coloring of $\overline{\text{int}}(C)$, so we are done.

Given a triangulated plane graph G represented via adjacency lists, we can apply Algorithm 3.2 as follows. First, create length n integer arrays for c and S, initializing each entry to zero. Let $C = v_1, v_2, v_3$ be the outer triangle of G, labeled in clockwise order. Color $c(v_1) \leftarrow 1$, $c(v_2) \leftarrow 2$, and $c(v_3) \leftarrow 2$. Iterate through $N(v_1)$ and mark $S(v) \leftarrow 1$ for each neighbor v of v_1. Then call *Path3Color*$(v_1, {}^{v_1}v_3, 1)$ with G, c, and S.

During Algorithm 3.2, $N(z)$ is iterated exactly twice for each $z \in G$: once to locate ${}^za$ at the start of Loop 2, and once in Loops 2.1 and 2.2. Thus its time complexity is $\mathcal{O}(n)$.

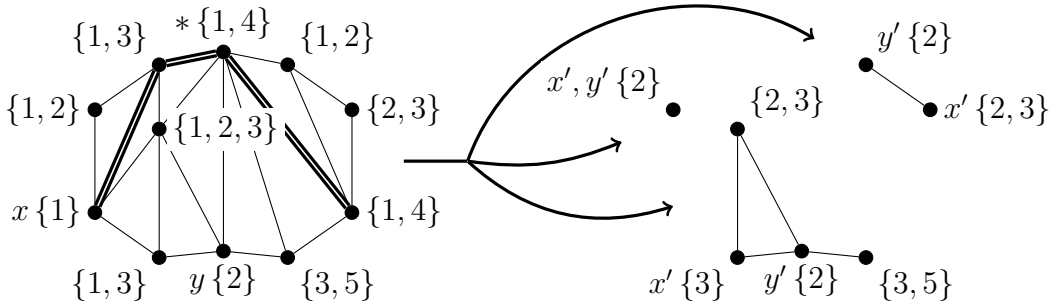


Figure 4: Coloring and removing a path P as in Hartman's proof [7, Lemma 4.2].

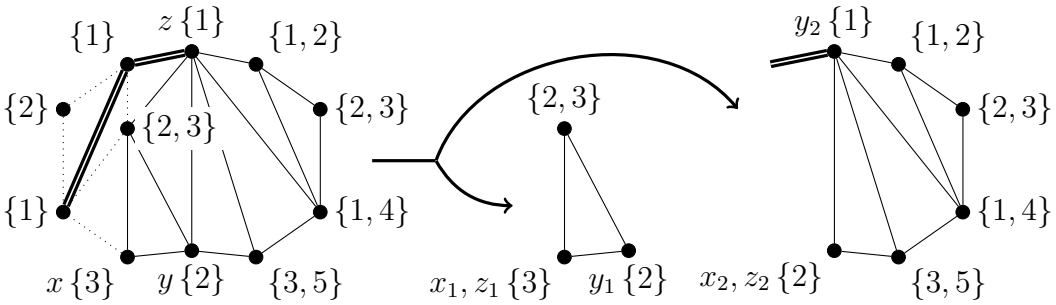


Figure 5: Coloring and removing a path P as in the proof of Lemma 4.1 (Algorithm 4.2).

4 Path List-Coloring

In this section we describe a linear time algorithm to path color a plane graph such that each vertex receives a color from a specified list of three colors, based on Hartman's constructive proof of Theorem 1.2.

Hartman's algorithm takes as its input a weakly triangulated plane graph G with outer face C, a list assignment L, and vertices $x, y \in C$. It assumes $|L(v)| \geq 1$ if $v \in \{x, y\} \cup S$, $|L(v)| \geq 2$ if $v \in C - \{x, y\} \cup S$, and $|L(v)| \geq 3$ if $v \in G - C$, where S is the set of cut-vertices separating x from y. An induced path P is colored with $d \in L(x)$, proceeding from x towards y along $C[x, y]$. Starting from the single vertex $P = x$, the path P is constructed as follows: let z be the end of P opposite x, and $T_z = \{v \in N(z) \cap V(C[z, y]) : d \in L(v)\}$; if $T_z \neq \emptyset$, then the $v \in T_z$ closest to y in $C[z, y]$ is colored and added to P; this process is then repeated (with $z = v$) until $T_z = \emptyset$. The algorithm is then recursively applied to each connected component of $G - P$, using the list assignment L' defined by $L'(v) = L(v) - \{d\}$ if $v \in N(P)$, and $L'(v) = L(v)$ if $v \notin N(P)$. See Figure 4 for an example.

In Lemma 4.1 below, we provide a different proof of Hartman's result, organized in such a way that it immediately describes a linear time algorithm for graphs represented as in Section 2. If $x = z$, then our G, x, y match Hartman's, but we assume that G is biconnected (so $S = \emptyset$); we follow a similar procedure to color a path P clockwise from x

along $C[x, y]$, but we consider one edge at a time, and simultaneously remove P from G as it is colored. If $x \neq z$, then z is the farthest clockwise end of the path P being colored; the rest of P (if any) has already been removed, and our x serves as Hartman's x'.

The one major difference between our algorithm and Hartman's occurs in Case 3.2.3.2 of the proof of Lemma 4.1 below. If we encounter a chord edge $zv \in E(G) - E(C)$ (here z is the current end of P) with $v \in C[z, y]$ and $L(z) \cap L(v) = \emptyset$ (i.e. $d \notin L(v)$, where d is the color of P), then instead of continuing to search for neighbors of z in $C[z, v]$ that we might add to P, we apply the inductive hypothesis such that z now serves as "y". In Figure 4, the edge from the vertex labeled $*$ to y is such a chord: our algorithm would halt P at $*$, and divide the remaining subgraph at the chord, see Figure 5. For a general illustration of this case, see Figure 8.

Given a coloring c of a graph G, and $v \in G$, we define $\deg_c(v)$ to be $\deg(v)$ in the subgraph induced by $c(v)$'s color class, i.e. the number of neighbors that share v's color.

Lemma 4.1. *Let G be a biconnected, weakly triangulated plane graph with outer cycle C. Let $x, y, z \in C$ be vertices (not necessarily distinct) such that $z \in C[x, y]$. Let L be a list assignment of G such that*

$$|L(v)| = 1 \text{ for } v \in \{x, y, z\},$$
$$|L(v)| \geq 2 \text{ for } v \in C - \{x, y, z\},$$
$$|L(v)| \geq 3 \text{ for } v \in G - C,$$

and if $v \in C[x, z] - z$, then $L(v) \cap L(z) = \emptyset$. There exists a path L-list-coloring c of G such that $\deg_c(x) \leq 1$, $\deg_c(y) \leq 1$, and $\deg_c(z) \leq 1$. Moreover, if $z = y$, or if $C[z, y] = z, y$ and $L(z) \cap L(y) = \emptyset$, then $\deg_c(z) = 0$.

Proof. Suppose that $m = |E(G)| \leq 1$. We define the path L-list-coloring c of $G = C$ as follows. Let $c(x) \in L(x)$ and $c(y) \in L(y)$ be the color in each respective list. If $m = 0$, then $|V(G)| = 1$ and c trivially satisfies the lemma. Otherwise $m = 1$ and $|V(G)| = 2$. If $x = y$, then the vertex $v \in G - y$ satisfies $|L(v)| \geq 2$, so define $c(v) \in L(v) - L(y)$. Note that $\deg_c(v) \leq 1$ for each $v \in G$, with $\deg_c(z) = 1$ only if $z \neq y$ and $L(z) = L(y)$.

We proceed by induction on m, the number of edges. Let $u_1, u_2, \ldots, u_k$ label the neighbors of z in counter-clockwise order such that u_1 is the vertex immediately counter-clockwise from z around C, and u_k is the vertex clockwise from z around C. In each of the subsequent cases we will apply the lemma separately to one or two subgraphs G_1, G_2 of G, with corresponding outer face vertices labeled $x_i, y_i, z_i \in G_i$ for $i = 1, 2$.

Case 1. Suppose that $u_1 = y$. Note that $z = x$ since $z \in C[x, y]$. Therefore $V(C[z, y]) = V(C)$, so $|V(C[z, y])| \geq 3$ and $C[z, y] \neq z, y$. Apply the lemma to find a path L-list-coloring c of $G_1 = G$ with $x_1 = y$, $y_1 = x$, and $z_1 = y$, see Figure 6 (left).

Case 2. Suppose that $u_1 \neq y$ and $z = x$. Then $u_1 \in C - C[x, z]$ and $|L(u_1)| \geq 2$. Pick $d \in L(u_1) - L(z)$. Define $L'(u_1) = \{d\}$ and $L'(v) = L(v)$ for each $v \in G - u_1$. Find a path L'-list-coloring of $G_1 = G$ with $x_1 = u_1$, $y_1 = y$, and $z_1 = z$, see Figure 6 (middle).

Case 3. Suppose that $u_1 \neq y$ and $z \neq x$. Our strategy will be to apply the inductive hypothesis to the *blocks* (maximal biconnected subgraphs) of $G - zu_1$. Because $u_1 \in$

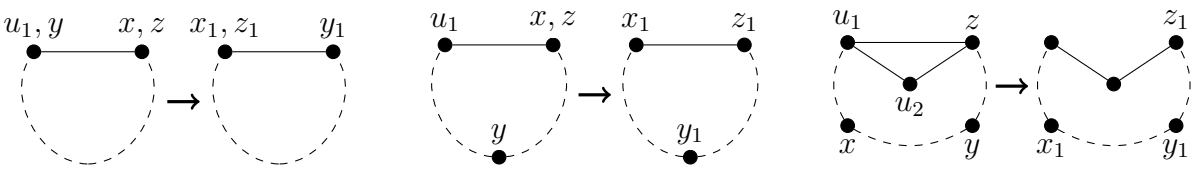


Figure 6: The proof of Lemma 4.1, Case 1 (left), Case 2 (middle), and Case 3.1 (right).

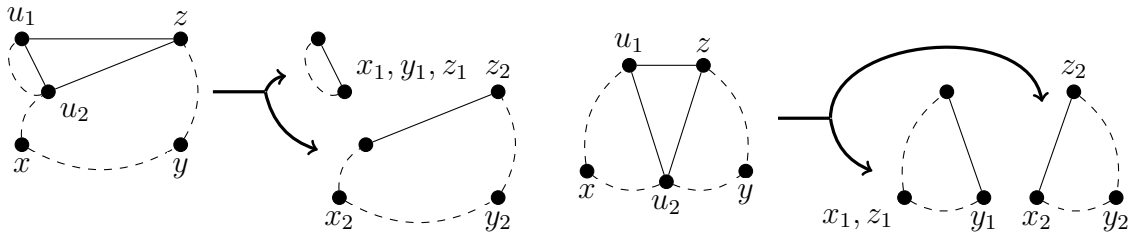


Figure 7: The proof of Lemma 4.1, Case 3.2.1 (left) and Case 3.2.2 (right).

$C[x, z]$, we have $L(u_1) \cap L(z) = \emptyset$. Thus any path L-list-coloring of $G - zu_1$ is also a path L-list-coloring of G. There are three cases to consider, see Figures 6 (right), 7, and 8.

Case 3.1. Suppose that $u_2 \in G - C$. Define $L'(u_2) = L(u_2) - L(z)$ and $L'(v) = L(v)$ for each $v \in G - u_2$. Note that $|L'(u_2)| \geq |L(u_2)| - 1 \geq 2$. Let C_1 be the outer face of $G_1 = G - zu_1$, and observe that $C_1[x, z]$ is the union of $C[x, z] - u_1z$ with the path u_1, u_2, z. Since $|L'(u_2)| \geq 2$ and $L'(u_2) \cap L'(z) = \emptyset$, we may apply the inductive hypothesis to find a path L'-list-coloring of G_1 with $x_1 = x$, $y_1 = y$, $z_1 = z$.

Case 3.2. Suppose that $u_2 \in C$. Observe that u_2 is a *cut-vertex* of $G - zu_1$, i.e. $G - zu_1 - u_2$ is disconnected. Define $C_1 = C[u_2, u_1] \cup u_1u_2$ and $C_2 = C[z, u_2] \cup zu_2$. Let $G_1 = \overline{\text{int}}(C_1)$ and $G_2 = \overline{\text{int}}(C_2)$. The subgraphs G_1 and G_2 are the two blocks of $G - zu_1$.

In each subsequent case we will apply the inductive hypothesis to find a path L-list-coloring c_1 of G_1 and a path L-list-coloring c_2 of G_2 such that $c_1(u_2) = c_2(u_2)$. Let c be the L-list-coloring of G defined by $v \mapsto c_1(v)$ for $v \in G_1$ and $v \mapsto c_2(v)$ for $v \in G_2$. Observe that $\deg_c(v) = \deg_{c_1}(v)$ for each $v \in G_1 - u_2$, $\deg_c(v) = \deg_{c_2}(v)$ for each $v \in G_2 - u_2$, and $\deg_c(u_2) = \deg_{c_1}(u_2) + \deg_{c_2}(u_2)$. To show that c is a path L-list-coloring of G it suffices to show that $\deg_c(u_2) \leq 2$.

Case 3.2.1. Suppose that $u_2 \in C[x, z]$. Observe that $x, y, z \in C_2$. Apply the inductive hypothesis to find a path L-list-coloring c_2 of G_2 with $x_2 = x$, $y_2 = y$, and $z_2 = z$. Define $L'(u_2) = \{c_2(u_2)\}$ and $L'(v) = L(v)$ for each $v \in G_1 - u_2$. Find a path L'-list-coloring c_1 of G_1 with $x_1 = y_1 = z_1 = u_2$. Since $\deg_{c_1}(u_2) = 0$, c is a path L-list-coloring of G. Moreover, since $\deg_c(v) = \deg_{c_2}(v)$ for all $v \in G_2$, the $\deg_c$ constraints for x, y, z follow from the inductive hypothesis.

Case 3.2.2. Suppose that $u_2 \in C[y, x] - x - y$. Pick $d \in L(u_2) - L(z)$. Define $L'(u_2) = \{d\}$ and $L'(v) = L(v)$ for each $v \in G - u_2$. Observe that $x \in C_1$ and $z, y \in C_2$. Apply the inductive hypothesis to find a path L'-list-coloring c_1 of G_1 with $x_1 = z_1 = x$ and $y_1 = u_2$. Then find a path L'-list-coloring c_2 of G_2 with $x_2 = u_2$, $y_2 = y$, and

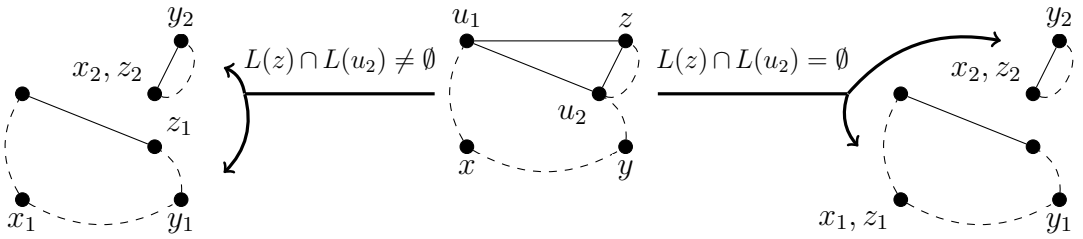


Figure 8: The proof of Lemma 4.1, Case 3.2.3.

$z_2 = z$. Since $\deg_{c_1}(u_2) \leq 1$ and $\deg_{c_2}(u_2) \leq 1$, c is a path L-list-coloring of G. Since $\deg_c(x) = \deg_{c_1}(x_1)$, $\deg_c(y) = \deg_{c_2}(y_2)$, and $\deg_c(z) = \deg_{c_2}(z_2)$, the $\deg_c$ constraints again follow from the inductive hypotheses.

Case 3.2.3. Suppose that $u_2 \in C[z, y]$. Note that $z \neq y$. There are two cases to consider, see Figure 8.

Case 3.2.3.1. Suppose that $L(z) \cap L(u_2) \neq \emptyset$. Then either $u_2 = y$ and $L(z) = L(y)$, or $C[z, y] \neq z, y$. Define $L'(u_2) = L(z)$, and $L'(v) = L(v)$ for each $v \in G - u_2$. Find a path L'-list-coloring c_1 of G_1 with $x_1 = x$, $y_1 = y$, and $z_1 = u_2$. Then find a path L'-list-coloring c_2 of G_2 with $x_2 = u_2$, $y_2 = z$, and $z_2 = u_2$. Since $\deg_{c_1}(u_2) \leq 1$ and $\deg_{c_2}(u_2) = 1$, it follows that c is a path L-list-coloring of G. If $u_2 = y$, then $z_1 = y_1 = y$ and $\deg_{c_1}(y) = 0$; thus $\deg_c(y) = \deg_{c_2}(u_2) = 1$. If $u_2 \neq y$, then $\deg_c(y) = \deg_{c_1}(y_1) \leq 1$. In either case, $\deg_c(z) = \deg_{c_2}(y_2) = 1$ and $\deg_c(x) = \deg_{c_1}(x_1) \leq 1$.

Case 3.2.3.2. Suppose that $L(z) \cap L(u_2) = \emptyset$. Find a path L-list-coloring c_1 of G_1 with $x_1 = x$, $y_1 = y$, and $z_1 = x$. Define $L'(u_2) = \{c_1(u_2)\}$ and $L'(v) = L(v)$ for each $v \in G_2 - u_2$. Then find a path L'-list-coloring c_2 of G_2 with $x_2 = u_2$, $y_2 = z$, and $z_2 = u_2$. Because $C_2[z_2, y_2] = z_2, y_2$ and $L'(z_2) \cap L'(y_2) = \emptyset$, we must have $\deg_{c_2}(u_2) = \deg_{c_2}(z_2) = 0$. Thus c is a path L-list-coloring of G. Moreover, $\deg_c(x) = \deg_{c_1}(x_1) \leq 1$, $\deg_c(y) = \deg_{c_1}(y_1) \leq 1$, and $\deg_c(z) = \deg_{c_2}(y_2) \leq 1$. If $C[z, y] = z, y$, then $G_2 = z, y$ and $\deg_c(z) = 0$, since $c(z) \notin L(y)$. $\square$

Let G be a planar graph and L a list assignment such that $|L(v)| \geq 3$ for each $v \in G$. Compute a planar embedding and add edges to produce a triangulated plane graph G'. Pick $u \in G'$ on the outer face and $d \in L(u)$. Define the list assignment L' by $L'(u) = \{d\}$ and $L'(v) = L(v)$ for $v \in G' - u$. By Lemma 4.1 there exists a path L'-list-coloring of G', and thus a path L-list-coloring of G. So Theorem 1.2 follows from Lemma 4.1.

The proof of Lemma 4.1 describes a linear time path list-coloring algorithm, see Algorithm 4.2. Given $v \in G$ and any entry ${}^v u$ in $N(v)$, we'll assume the entry $\text{back}({}^v u) = {}^u v$ in $N(u)$ may be computed in $\mathcal{O}(1)$ time; see augmented adjacency lists in Section 2.

Algorithm 4.2: *Path3Choose*(x, y, z)

Input. Let C be a cycle or length 1 path in a weakly triangulated plane graph G represented via augmented adjacency lists. Let $x, y, z \in C$ be such that $z \in C[x, y]$.

Let L be a length n array of lists of colors such that for $v \in \{x, y, z\}$ each list $L(v)$ has length one, for $v \in C - \{x, y, z\}$ each list $L(v)$ has length two or three, and for $v \in \text{int}(C)$

each list $L(v)$ has length three. Furthermore, for each $v \in C[x,z] - z$, assume that $L(v)$ does not contain the color in $L(z)$.

Let A, B be length n arrays such that for each vertex $v \in C$ we have $A(v) = {}^{v}a$ and $B(v) = {}^{v}b$, where a is the vertex immediately clockwise from v around C, and b is the vertex immediately counter clockwise from v around C.

Let S be a length n integer array such that, for $v \in \overline{\text{int}}(C)$, $S(v) \neq 0$ if and only if $v \in C$, and $S(v) = S(x)$ if and only if $v \in C[x,z] - z$ (or $v = x = z$).

We'll write $m_C = |E(\overline{\text{int}}(C))|$ and $\max(S) = \max_{v \in G} S(v)$. Define $\ell = \max(S) + r$, with r determined by the case below that the input satisfies: if $m_C = 1$ (Base case) or $z \neq x$ (Case 3), then $r = 0$; if $z = x$ and $C[y,x] \neq y, x$ (Case 2), then $r = 1$; otherwise $z = x$ and $C[y,x] = y, x$ (Case 1), and $r = 2$. We'll assume that $\ell < 2(m - m_C + 1) + 2$.

Let M be a length $2m + 2$ integer array such that if $v \in C$, then $M(S(v)) = S(y)$ if and only if $v \in C[z,y]$. Moreover, if $x \neq z$, then assume that $M(S(x)) = S(x)$. The purpose of the array M is described below in Case 3.2.3.2.

The concrete input will be G, L, A, B, S, M, along with the tuple (x, y, z).

Output. For each $v \in \overline{\text{int}}(C)$, all but one color will be removed from the list $L(v)$. The remaining color in each list will represent a path L-list-coloring of $\overline{\text{int}}(C)$ such that $\deg_c(x) \leq 1$, $\deg_c(y) \leq 1$, and $\deg_c(z) \leq 1$. If $z = y$, or if $C[z,y] = z, y$ and $L(z)$ did not contain the color in $L(y)$, then $\deg_c(z) = 0$. We'll also guarantee $\max(S) \leq \ell + 2(m_C - 1)$.

Procedure. Let ${}^{z}u_1 := B(z)$. Our strategy will follow the proof of Lemma 4.1.

Base case. If $A(z) = B(z)$, then $C = z, u_1$ is a length 1 path. If $u_1 \neq x$ and $u_1 \neq y$, remove the color in $L(z)$ from $L(u_1)$, if it exists, then remove further colors until one color remains. Note that $m_C = 1$ and $\max(S) = \ell + 2(m_C - 1)$ by definition.

Recursive case. Suppose that $A(z) \neq B(z)$. Let ${}^{z}u_2 := \text{next}({}^{z}u_1)$. It follows from the definition of ℓ that $\max(S) \leq \ell + 2(m_C - 1)$ after Cases 1, 2, and 3.1, so we will only argue why the bound holds after Case 3.2. It may be useful to see Figures 6, 7, and 8.

Case 1. Suppose that $u_1 = y$. Then $z = x$, since $z \in C[x,y]$. Set $S(x) \leftarrow \max(S) + 1$ and $S(y) \leftarrow S(x)$. Assign $M(S(x)) \leftarrow S(x)$. Call *Path3Choose*(y, x, y).

Case 2. Suppose that $z = x$ and $u_1 \neq y$. Remove the color in $L(z)$ from $L(u_1)$, if it exists, then remove further colors until only one color remains. Assign $S(u_1) \leftarrow \max(S)+1$ and $M(S(u_1)) \leftarrow S(u_1)$, and call *Path3Choose*$(u_1, y, z)$.

Case 3. Suppose that $z \neq x$ and $u_1 \neq y$. Let ${}^{u_1}z := \text{back}({}^{z}u_1)$ and ${}^{u_2}z := \text{back}({}^{z}u_2)$. Remove the edge zu_1 by assigning $A(u_1) \leftarrow \text{back}(\text{next}({}^{u_2}z))$ and $B(z) \leftarrow \text{next}({}^{z}u_1)$.

Case 3.1. Suppose that $S(u_2) = 0$, i.e. $u_2 \in \text{int}(C)$. Assign $S(u_2) \leftarrow S(x)$, and initialize $B(u_2) \leftarrow {}^{u_2}u_1 = \text{next}({}^{u_2}z)$ and $A(u_2) \leftarrow {}^{u_2}z$, see Figure 6 (right). Remove the color in $L(z)$ from $L(u_2)$. Call *Path3Choose*(x, y, z).

Case 3.2. Suppose that $S(u_2) \neq 0$, i.e. $u_2 \in C$. Just as in Case 3.2 of the proof of Lemma 4.1, we will separately consider the two blocks G_1 and G_2 of $\overline{\text{int}}(C) - zu_1$. Note that $m_C - 1 = |E(G_1)| + |E(G_2)|$.

As we've already removed the edge zu_1 by adjusting $B(z)$ and $A(u_1)$, it remains to "split" the neighborhood of u_2 at the edge u_2z such that the recursive call on G_1 considers only the neighbors of u_2 counter-clockwise from z, and the call on G_2 considers z and all neighbors clockwise from z; refer to Figure 7 and Figure 8.

Let ${}^{u_2}a := A(u_2)$ and ${}^{u_2}b := B(u_2)$. To represent G_1, we will assign $A(u_2) \leftarrow {}^{u_2}a$ and $B(u_2) \leftarrow \text{next}({}^{u_2}z)$. To represent G_2, we'll assign $A(u_2) \leftarrow {}^{u_2}z$ and $B(u_2) \leftarrow {}^{u_2}b$

Case 3.2.1. Suppose that $S(u_2) = S(x)$, i.e. $u_2 \in C[x, z]$. Call *Path3Choose*(x, y, z) on G_2 (by modifying $A(u_2)$ and $B(u_2)$ as above). Afterwards $\max(S) \leq \ell + 2|E(G_2)| - 2$. Assign $S(u_2) \leftarrow \max(S) + 1$ and $M(S(u_2)) \leftarrow S(u_2)$, and call *Path3Choose*$(u_2, u_2, u_2)$ on G_1. Finally, observe that $\max(S) \leq \ell + 2|E(G_2)| - 2 + 1 + 2|E(G_1)| - 1 < \ell + 2(m_C - 1)$.

Case 3.2.2. Suppose that $S(u_2) \neq 0$, but $S(u_2) \neq S(x)$ and $M(S(u_2)) \neq S(y)$. In other words, suppose that $u_2 \in C[y, x] - x - y$. Remove the color in $L(z)$ from $L(u_2)$, if it exists, then remove further colors until one remains. Set $S(u_2) \leftarrow S(x)$. Note $M(S(v)) = S(u_2)$ for each $v \in C_1[x, u_2]$. Call *Path3Choose*(x, u_2, x) on G_1. Afterwards $\max(S) \leq \ell + 2|E(G_1)|$. Set $S(u_2) \leftarrow \max(S) + 1$, $M(S(u_2)) \leftarrow S(u_2)$, and call *Path3Choose*$(u_2, y, z)$ on G_2. Finally, observe that $\max(S) \leq \ell + 2|E(G_1)| + 1 + 2|E(G_2)| - 2 < \ell + 2(m_C - 1)$.

Case 3.2.3. Suppose that $M(S(u_2)) = S(y)$, i.e. $u_2 \in C[z, y]$. There are two cases to consider, see Figure 8.

Case 3.2.3.1. Suppose that the color in $L(z)$ is also in $L(u_2)$. Remove all colors from $L(u_2)$ except for the color in $L(z)$. Call *Path3Choose*(x, y, u_2) on G_1. Afterwards $\max(S) \leq \ell + 2|E(G_1)| - 2$. Set $S(z) \leftarrow \max(S) + 1$, $S(u_2) \leftarrow S(z)$, and $M(S(z)) \leftarrow S(z)$. Call *Path3Choose*(u_2, z, u_2) on G_2. Finally, observe that $\max(S) \leq \ell + 2|E(G_1)| - 2 + 1 + 2|E(G_2)| - 1 < \ell + 2(m_C - 1)$.

Case 3.2.3.2. Suppose the color in $L(z)$ is not in $L(u_2)$. Set $M(S(x)) \leftarrow S(y)$. Call *Path3Choose*(x, y, x) on G_1. Afterwards $\max(S) \leq \ell + 2|E(G_1)|$. Set $S(z) \leftarrow \max(S) + 1$, $S(u_2) \leftarrow S(z)$, and $M(S(z)) \leftarrow S(z)$. Call *Path3Choose*(u_2, z, u_2) on G_2. Observe that $\max(S) \leq \ell + 2|E(G_1)| + 1 + 2|E(G_2)| - 1 = \ell + 2(m_C - 1)$.

Note that this is the only case where $M(S(v))$ is set a value other than $S(v)$. Setting $M(S(x)) \leftarrow S(y)$ effectively "joins" $C[x, u_1]$ onto $C[u_2, y]$ to represent $C_1[z_1, y_1]$ in G_1, see Figure 8. Without the array M, we'd need to re-mark each $v \in C[x, u_1]$ with $S(v) \leftarrow S(y)$.

Let G be a triangulated plane graph represented by augmented adjacency lists. Let L be a length n array of color lists of length 3, representing a list assignment of G. We may path L-color G using Algorithm 4.2 as follows.

Create length n arrays A, B and initialize $A(v)$ and $B(v)$ for each $v \in C$ on the outer face of G. Create a length n integer array S and initialize $S(v) \leftarrow 1$ for each $v \in C$ and $S(v) \leftarrow 0$ for each $v \in G - C$. Pick any $u \in C$, assign $S(u) \leftarrow 2$, and remove all but one color from $L(u)$. Create a length $2m + 2$ array of integers M and initialize $M(i) \leftarrow i$ for each $i \in \{0, 1, 2\}$. Finally, call *Path3Choose*(u, u, u) with G, L, A, B, S, and M. The remaining color in each list $L(v)$ will represent a path L-list-coloring of G.

Each case in Algorithm 4.2 performs a fixed number of constant time operations and makes up to two recursive calls. Case 3 removes an edge from the subgraph(s) under consideration. Case 1 makes a recursive call with an input that satisfies Case 2, and Case 2 makes a recursive call with an input that satisfies Case 3. Therefore the total number of recursive calls made, and thus overall the time complexity, is $\mathcal{O}(n)$.

See Figures 9 and 10 for a full example.

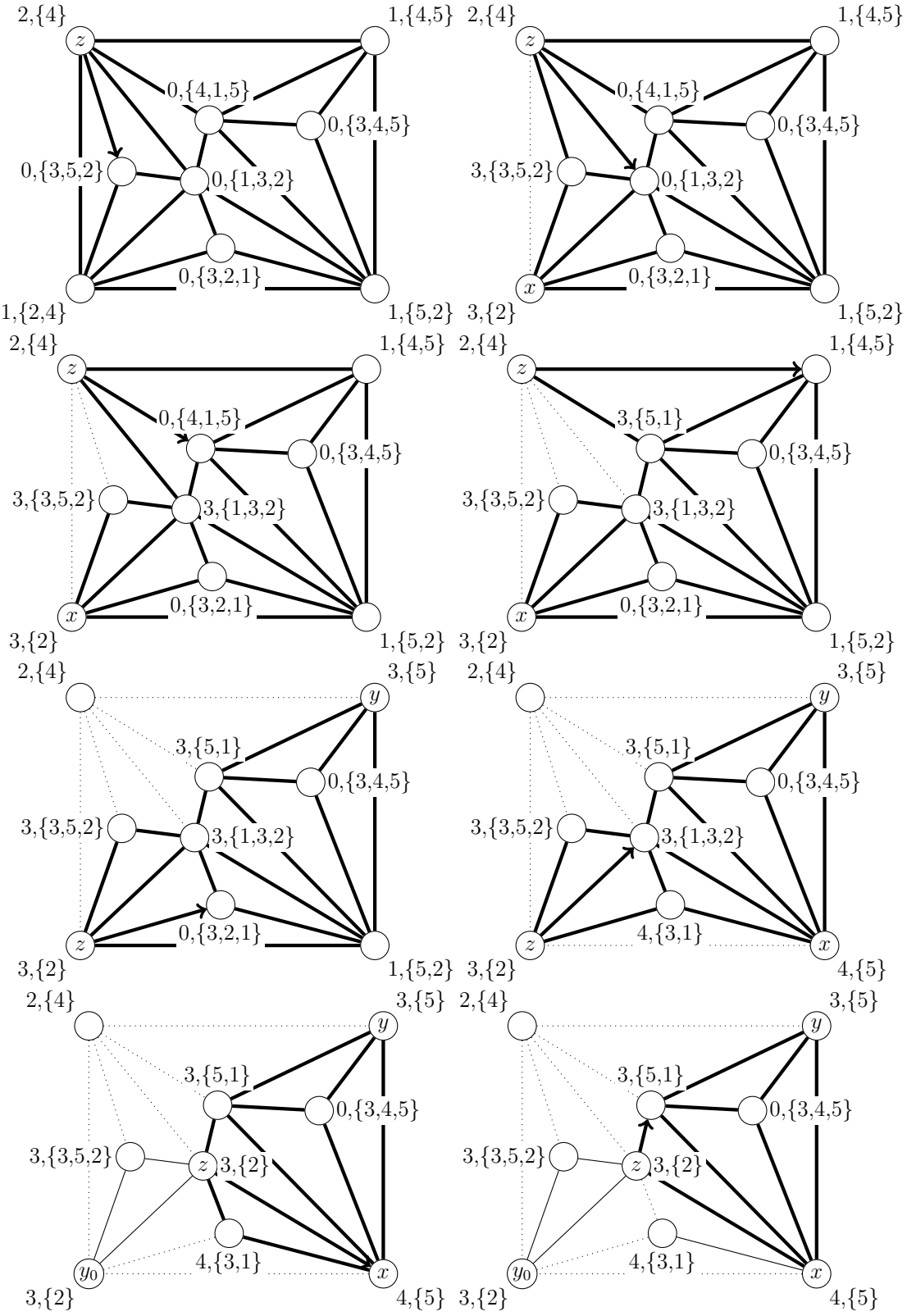


Figure 9: An example of Algorithm 4.2. Each vertex $v \in G$ is labeled with $S(v), L(v)$.

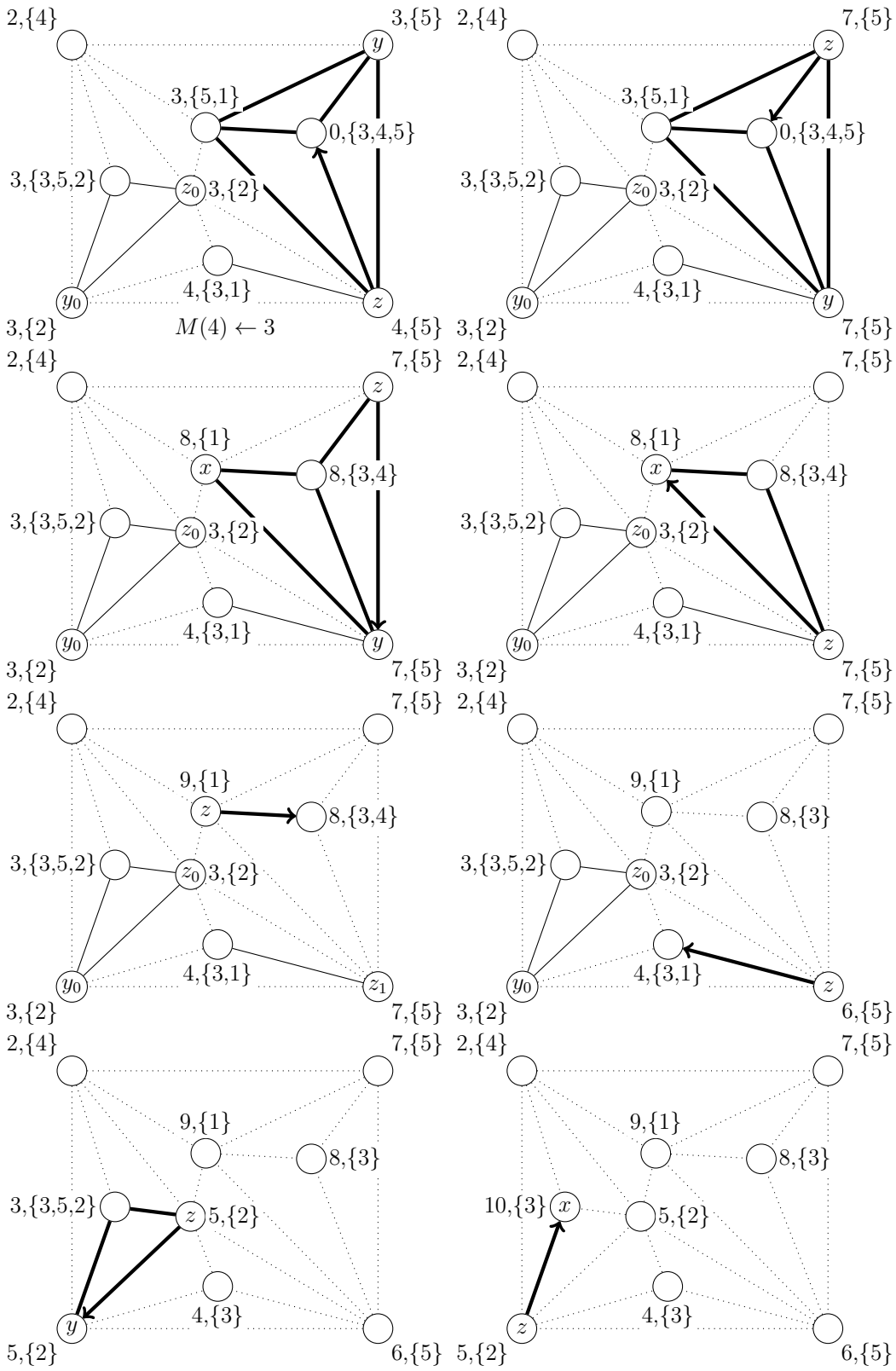


Figure 10: An Algorithm 4.2 example, continued from Figure 9.

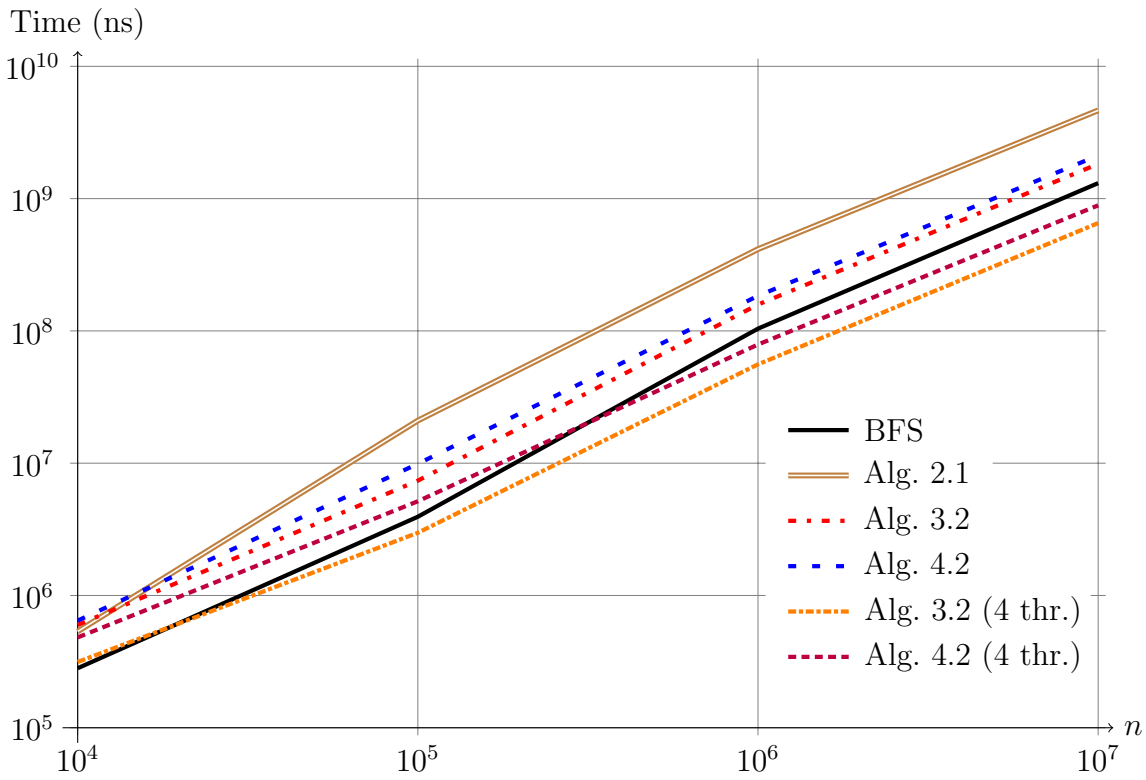


Figure 11: Average time per plane triangulation on a uniform random sample.

5 Experimental Performance and Parallelism

A C implementation of each algorithm is available [3]. For each i from $4 \ldots 7$, a uniform random sample of 100 plane triangulations of order 10^i was generated using the method of Poulalhon and Schaeffer [10]. Each algorithm, as well as a baseline $\mathcal{O}(n)$ breadth-first search (BFS), was run on each graph in the sample set; see Figure 11.

In Algorithm 3.2 and Algorithm 4.2, it is possible to maintain a stack of independent recursive frames, and to have a pool of threads execute them concurrently. The amount of parallelism possible using this method greatly varies depending on the graph structure, but a 2-3x average performance gain (2.8x for Alg. 3.2, and 2.4x for Alg. 4.2) was observed when using 4 threads to path color large random triangulations, see Figure 11.

Our parallel implementation of Algorithm 3.2 maintains a shared lock-guarded stack which all threads may push/pop recursive frames to/from.

Algorithm 4.2 is trickier to adapt to parallel execution: the pairs of recursive calls in Case 3.2 rely on a sequential execution order. If we save a local copy of $S(v), A(v), B(v)$ for each $v \in \{x, y, z\}$ alongside each frame, then the recursive calls in Case 3.2.1 with $u_2 = x$, Case 3.2.2, Case 3.2.3.1, and Case 3.2.3.2 with $u_2 = y$ become order independent. However, in Case 3.2.1 with $u_2 \neq x$, and Case 3.2.3.2 with $u_2 \neq y$, the second call will always depend on the color of u_2 (the remaining color in $L(u_2)$) computed by the first.

Our parallel implementation of Algorithm 4.2 maintains a shared object pool of frames and a shared stack of references to frames in the pool. Frames are added to the pool in the usual course of the algorithm, but only pushed to the stack if they are ready to execute. For each $v \in G$, we store an optional reference to a frame that will become ready when v has been colored. To allow for multiple frames to await the coloring of one vertex, each frame in the pool stores an optional reference to a "next frame in line" (i.e. an "intrusive" linked list). When a vertex v is colored, each frame waiting on v is pushed to the stack.

References

[1] J. Boyer and W. Myrvold, On the cutting edge: simplified $O(n)$ planarity by edge addition, *J. Graph Algorithms Appl.* **8** (2004), 241–273.

[2] I. Broere and C. M. Mynhardt, Generalized colorings of outerplanar and planar graphs, *Graph theory with applications to algorithms and computer science* (Kalamazoo, Mich., 1984), pp. 151–161, Wiley-Intersci. Publ., Wiley, New York, 1985.

[3] A. Bross, *Implementing path coloring algorithms on planar graphs*, Masters Project, University of Alaska, 2017, available from `http://github.com/permutationlock/path_coloring_bgl`.

[4] G. G. Chappell and C. Hartman, Path choosability of planar graphs, *The Electronic Journal of Combinatorics* **25** (2018), P4.33.

[5] G. Chartrand and H. V. Kronk, The point-arboricity of planar graphs, *J. London. Math. Soc.* **44** (1969), 612–616.

[6] W. Goddard, Acyclic colorings of planar graphs, *Discrete Math.* **91** (1991), no. 1, 91–94.

[7] C. M. Hartman, *Extremal Problems in Graph Theory*, Ph.D. Thesis, University of Illinois, 1997.

[8] J.E. Hopcroft and R.E. Tarjan, Efficient Planarity Testing, *Journal of the ACM* **21** (1974), 549-568

[9] K. S. Poh, On the linear vertex-arboricity of a planar graph, *J. Graph Theory* **14** (1990), no. 1, 73–75.

[10] D. Poulalhon and G. Schaeffer, Optimal Coding and Sampling of Triangulations, *Algorithmica* **46** (2006), 505–527

[11] R. Škrekovski, List improper colourings of planar graphs, *Combin. Probab. Comput.* **8** (1999), no. 3, 293–299.

[12] D. B. West, *Introduction to Graph Theory, 2nd ed.*, Prentice Hall, Upper Saddle River, NJ, 2000.